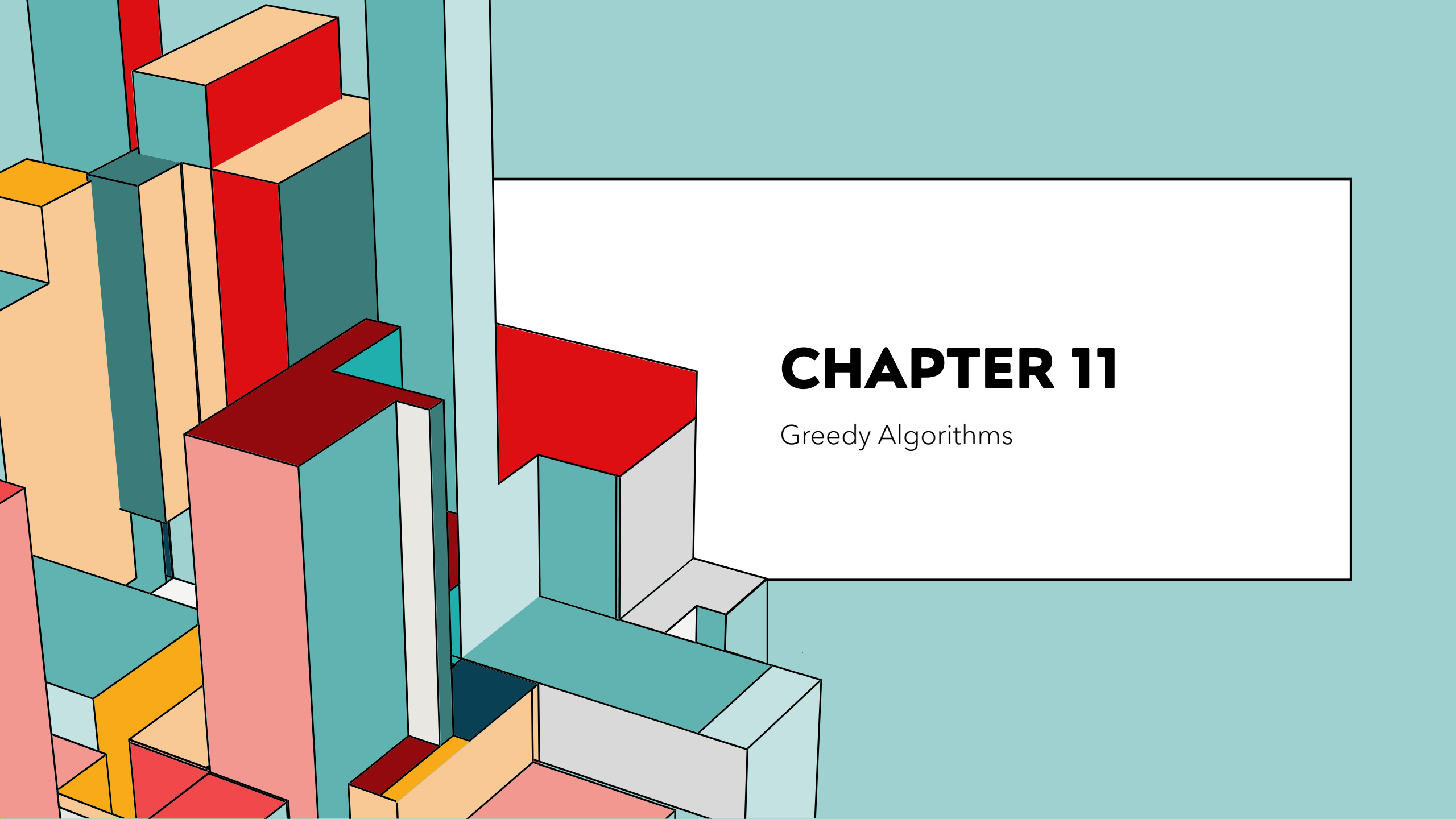




CHAPTER 11

Greedy Algorithms

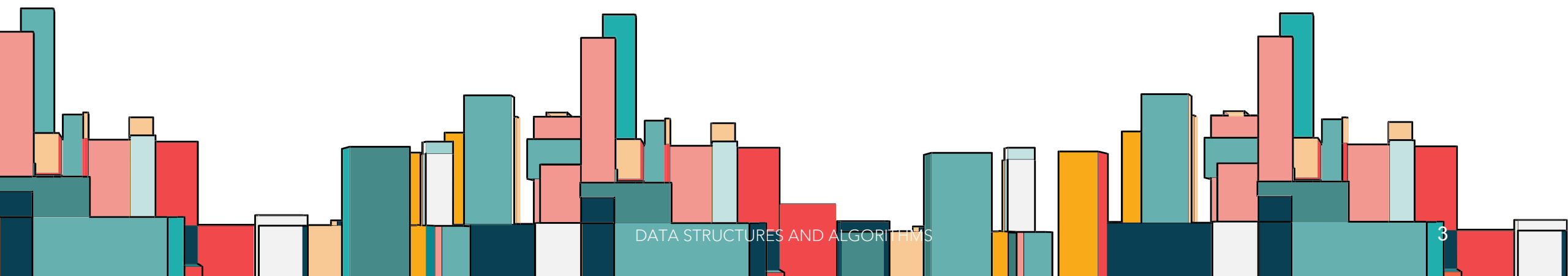
GREEDY ALGORITHMS

- Greedy = ละโมบ โลก
- Greedy Algorithm
 - อัลกอริทึมที่ทำการแก้ปัญหาโดยการเลือกทำสิ่งที่ให้ผลลัพธ์ที่ดีที่สุดในขณะนั้น
ทำเช่นนี้ไปเรื่อยๆ จนกว่าจะแก้ปัญหาได้สำเร็จ
 - เป็นแนวคิดในการแก้ปัญหาที่มักพบทั่วไป
 - ไม่รับประกันว่าผลลัพธ์จากแนวคิดนี้จะเป็นผลลัพธ์ที่ดีที่สุด

→ Optimal solution

ตัวอย่างปัญหาที่ใช้ GREEDY ALGORITHMS

- Coin exchange problem
- Classroom scheduling problem
- Knapsack problem
- Set-covering problem
- Traveling salesman problem



COIN EXCHANGE PROBLEM

- ปัญหาทอนเหรียญ
- สมมติว่าพนักงานขายมีเหรียญ 10, 5, 2 และ 1 บาท ไม่จำกัดจำนวน และพนักงานขายต้องการทอนเงินให้ลูกค้า โดยพยายามให้จำนวนเหรียญที่ทอนมีจำนวนน้อยๆ
- แนวคิดทั่วไป
 - เราจะทอนเงินด้วยเหรียญที่มีค่ามากที่สุดก่อน จนไม่สามารถทอนต่อได้ จึงเริ่มทอนด้วยเหรียญที่มีค่ารองลงมา ทำเช่นนี้ไปเรื่อยๆ จนกว่าจะทอนเงินได้ครบ -> นี่คือ greedy algorithm

COIN EXCHANGE PROBLEM (CONT.)

- พนักงานต้องการทอนเงินลูกค้า 29 บาท
 - ทอนเหรียญ 10 บาท 2 เหรียญ (รวม 20 บาท)
 - ทอนเหรียญ 5 บาท 1 เหรียญ (รวม 5 บาท)
 - ทอนเหรียญ 2 บาท 2 เหรียญ (รวม 4 บาท)
- พนักงานทอนเงินลูกค้า 29 บาทด้วยเหรียญ 5 เหรียญ ซึ่งเป็นคำตอบที่ดีที่สุดแล้ว (ใช้จำนวนเหรียญน้อยที่สุด) -> Optimal Solution
- ปกติแล้ว การใช้ Greedy algorithm ในการทอนเงิน จะได้คำตอบที่ดีที่สุดอยู่แล้ว

COIN EXCHANGE PROBLEM (CONT.)

- สมมุติให้มีเหรียญชนิด 10, 8, 5, 2 และ 1 บาท
- พนักงานต้องการทอนเงินลูกค้าเป็นเงิน 13 บาท ใช้ Greedy algorithm จะได้
 - ทอนเหรียญ 10 บาท 1 เหรียญ
 - ทอนเหรียญ 2 บาท 1 เหรียญ
 - ทอนเหรียญ 1 บาท 1 เหรียญ
 - รวมทอนได้ 3 เหรียญ
- เป็นคำตอบที่ดีที่สุดหรือไม่?

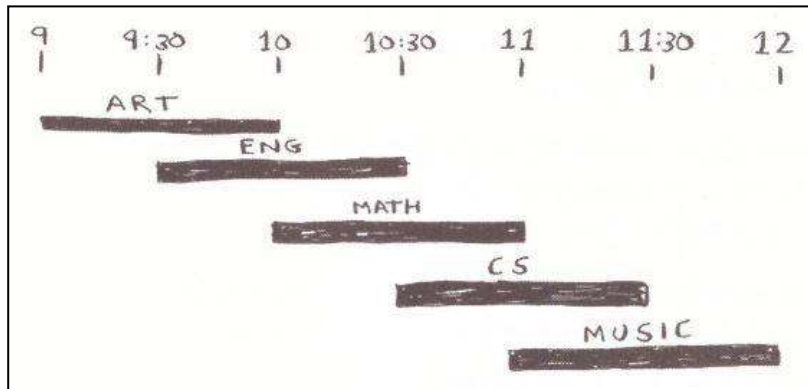
CLASSROOM SCHEDULING PROBLEM

- ปัญหาการจัดตารางเรียน
- สมมติให้มีรายวิชาที่เปิดให้นักเรียนได้ลงเรียนจำนวนมาก ซึ่งแต่ละรายวิชาอาจมีช่วงเวลาเรียนที่คาบเกี่ยวกัน
- นักเรียนจะเลือกเรียนวิชาต่างๆ อย่างไร เพื่อให้เรียนได้จำนวนวิชามากที่สุดเท่าที่จะทำได้

CLASSROOM SCHEDULING PROBLEM

CLASS	START	END
ART	9 AM	10 AM
ENG	9:30 AM	10:30 AM
MATH	10 AM	11 AM
CS	10:30 AM	11:30 AM
MUSIC	11 AM	12 PM

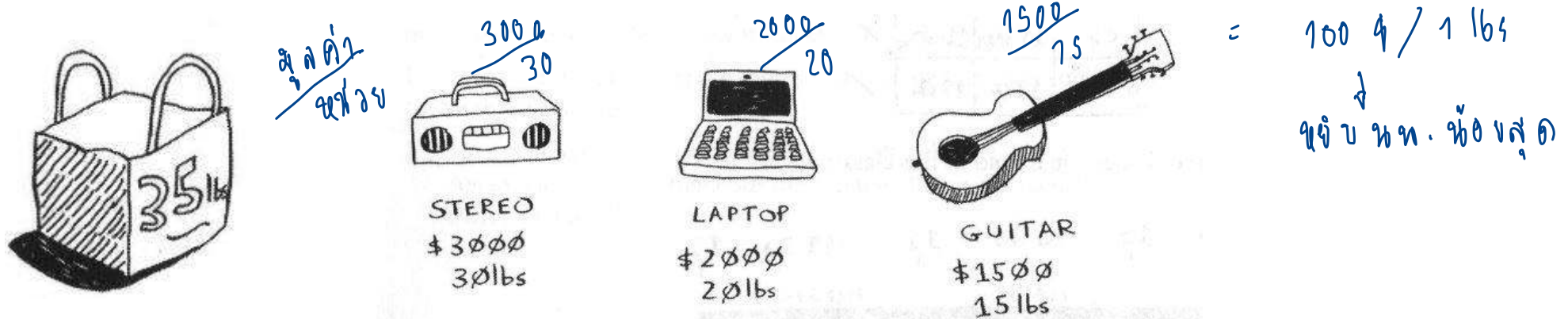
- แนวคิดในการลงเรียนวิชาให้ได้จำนวนมากที่สุด
 - เลือกเรียนวิชาที่เรียนเสร็จเร็วที่สุด
 - เวลาเรียนต้องไม่คาบเกี่ยวกัน
 - ทำเช่นนี้ไปเรื่อยๆ จนหมด



ART	9 AM	10 AM	✓
ENG	9:30 AM	10:30 AM	X
MATH	10 AM	11 AM	✓
CS	10:30 AM	11:30 AM	X
MUSIC	11 AM	12 PM	✓

KNAPSACK PROBLEM

- สมมติว่าคุณเป็นลูกค้าในร้านค้าแห่งหนึ่ง ตอนนี้ร้านมีโปรโมชั่นที่ทองให้คุณสามารถหยิบสินค้าที่ขึ้นก็ได้ลงในตะกร้าสินค้า ซึ่งกำหนดให้ตะกร้าไม่สามารถรับน้ำหนักเกิน 35 ปอนด์
- โจทย์คือ ให้คุณจะหยิบสินค้าอะไรลงในตะกร้าบ้าง เพื่อให้คุณได้ของที่มีมูลค่ามากที่สุด
→ เลือกของที่มีมูลค่าต่อหน่วยมากที่สุด

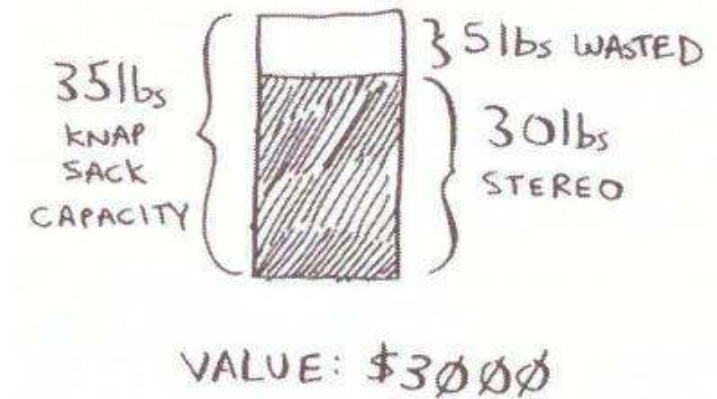


KNAPSACK PROBLEM (CONT.)

- แนวคิดในการแก้ปัญหานี้ด้วย Greedy algorithm
 - พยายามหยิบของที่แพงที่สุดก่อน ซึ่งเมื่อเอาของนั้นใส่ตะกร้าแล้วทำให้น้ำหนักตะกร้าไม่เกินน้ำหนักที่กำหนด
 - หยิบของที่มีราคารองลงมา และยังใส่ในตะกร้าได้
 - ทำเช่นนี้ไปเรื่อยๆ จนไม่สามารถหยิบของใส่ตะกร้าได้

KNAPSACK PROBLEM (CONT.)

- จากแนวคิดข้างต้น คุณจะสามารรถหยิบ Stereo ได้ 1 ชิ้น มูลค่า \$3000
- ในกรณีนี้ คำตอบจาก Greedy algorithm ไม่ใช่คำตอบที่ดีที่สุด



Optimal
Solution



SET-COVERING PROBLEM

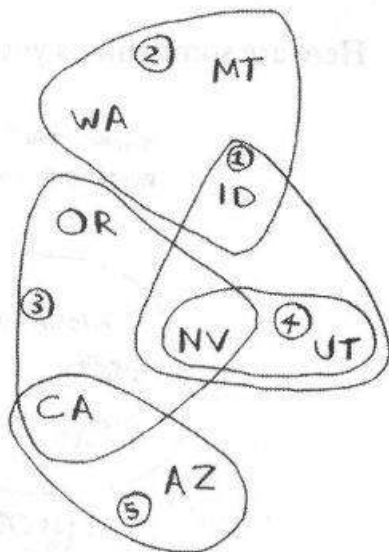
- เป็นปัญหาที่เกี่ยวข้องกับการหาเซตคำตอบที่ทำงานครอบคลุมกับเงื่อนไขที่กำหนด
- เป็นปัญหาที่ซับซ้อน โดยเฉพาะหากข้อมูลมีปริมาณมาก

SET-COVERING PROBLEM (CONT.)

RADIO STATION AVAILABLE IN

KONE	ID,NV,UT
KTWO	WA,ID,MT
KTHREE	OR,NV,CA
KFOUR	NV,UT
KFIVE	CA,AZ

...etc...



- สมมุติคุณเป็นเจ้าของสินค้าชนิดหนึ่งและต้องการทำโฆษณาผ่านสถานีวิทยุ โดยกำหนดให้มีข้อมูลชื่อสถานีและพื้นที่ที่สถานีนั้นครอบคลุม
- ปัญหา -> คุณจะต้องใช้เวลาโฆษณาของสถานีวิทยุใดบ้าง เพื่อให้โฆษณาเข้าถึงลูกค้าได้ครอบคลุมทุกเมือง
ใช้เวลาทั้งหมดที่สุก 6 ต่อกองคลุม

station = $\{k_1, k_2, k_3, k_4, k_5\} \rightarrow \text{PowerSet.}$

SET-COVERING PROBLEM (CONT.)

Possible Ans = $\{\emptyset, \{k_1\}, \{k_2\}, \dots, \{k_5\}, \{k_1, k_2\}, \dots, \{k_1, k_2, k_3, k_4, k_5\}\}$

- แนวคิดในการแก้ปัญหา
- หาเซตของสถานีวิทยุที่เป็นไปได้ทั้งหมด $\rightarrow$
 - 2^n เซตย่อย
 - n เป็นจำนวนสถานีวิทยุ
- เลือกเซตที่มีจำนวนสมาชิกน้อยที่สุด และครอบคลุมพื้นที่ครบทุกเมือง
- $O(2^n)$

2^5 คำตอบ
= 32 คำตอบ

NUMBER OF STATIONS	TIME TAKEN
5	3.2 sec
10	102.4 sec
32	13.6 years
100	4×10^{21} years

ใช้เวลา

1 คำตอบ $\approx$ 0.1 sec

n สถานี $\rightarrow 2^n$ คำตอบ
(เพิ่มขึ้นอย่างรวดเร็ว)

SET-COVERING PROBLEM (CONT.)

- แนวคิดในการแก้ปัญหา ด้วย Greedy algorithm
 - เลือกสถานีครอบคลุมเมืองที่ยังไม่ได้รับโฆษณาจำนวนมากที่สุดก่อน (อาจจะมีเมืองที่ได้รับโฆษณาจากสถานีที่เลือกไว้ก่อนหน้านี้ก็ได้)
 - ทำเช่นนี้ไปเรื่อยๆ จนกว่าทุกเมืองจะได้รับโฆษณา
- $O(n^2)$

$$n = \text{จ.ห. สถานี}$$

$$n = 5$$

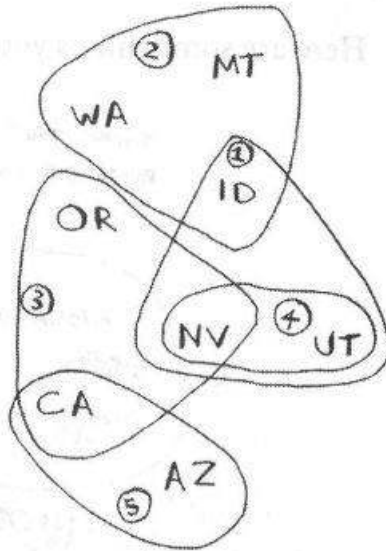
$$5 + 4 + 3 + 2 + 1$$

$$\left[\begin{array}{l} (n) + (n-1) + (n-2) + \dots \\ \rightarrow O(n) \end{array} \right.$$

SET-COVERING PROBLEM (CONT.)

$\{k_1, k_2, k_3, k_5\}$

RADIO STATION	AVAILABLE IN	
KONE	ID, NV, UT	3
KTWO	WA, ID, MT	3
KTHREE	OR, NV, CA	3
KFOUR	NV, UT	2
KFIVE	CA, AZ	2
...etc...		



- สมมุติคุณเป็นเจ้าของสินค้าชนิดหนึ่งและต้องการทำโฆษณาผ่านสถานีวิทยุ โดยกำหนดให้มีข้อมูลชื่อสถานีและพื้นที่ที่สถานีนั้นครอบคลุม
- ปัญหา -> คุณจะต้องใช้เวลาโฆษณาของสถานีวิทยุใดบ้าง เพื่อให้โฆษณาเข้าถึงลูกค้าได้ครอบคลุมทุกเมือง
ใช้เวลาทั้งหมดที่สั้นที่สุด ครอบคลุม

SET-COVERING PROBLEM (CONT.)

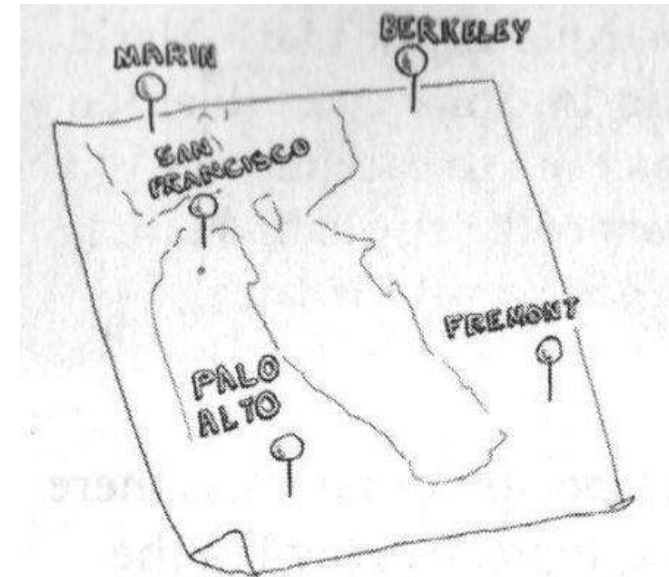
- แม้ว่า Greedy algorithm ไม่รับประกันว่าจะได้คำตอบที่ดีที่สุด (จำนวนสถานีน้อยที่สุด) แต่ประมวลผลเร็วกว่าแบบแรกมาก
- เรียกอีกอย่างว่า **Approximation algorithms**

NUMBER OF STATIONS	EXACT ALGORITHM	GREEDY ALGORITHM
5	3.2 sec	2.5 sec
10	102.4 sec	10 sec
32	13.6 yrs	102.4 sec
100	4×10^{21} yrs	16.67 min

เร็วแต่ไม่กำหนัดผลลัพธ์ที่ดีที่สุด

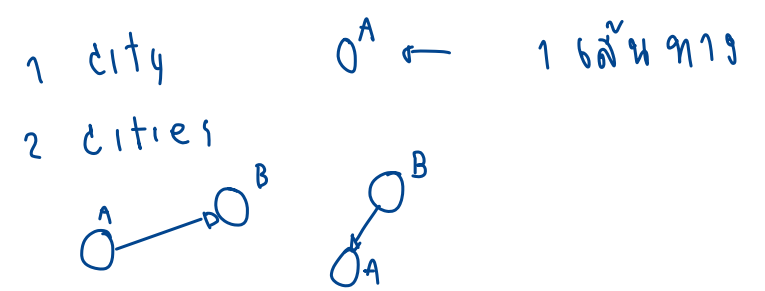
TRAVELING SALESMAN PROBLEM

- ปัญหาการเดินทางของเซลส์แมน
- สมมติให้คุณเป็นพนักงานขาย และต้องเดินทางไปขายสินค้าตามเมืองต่างๆ
- ปัญหา -> ทำอย่างไร คุณจะเดินทางไปตามเมืองต่างๆ ได้ครบทุกเมือง โดยเสียเวลาในการเดินทางน้อยที่สุด



TRAVELING SALESMAN PROBLEM

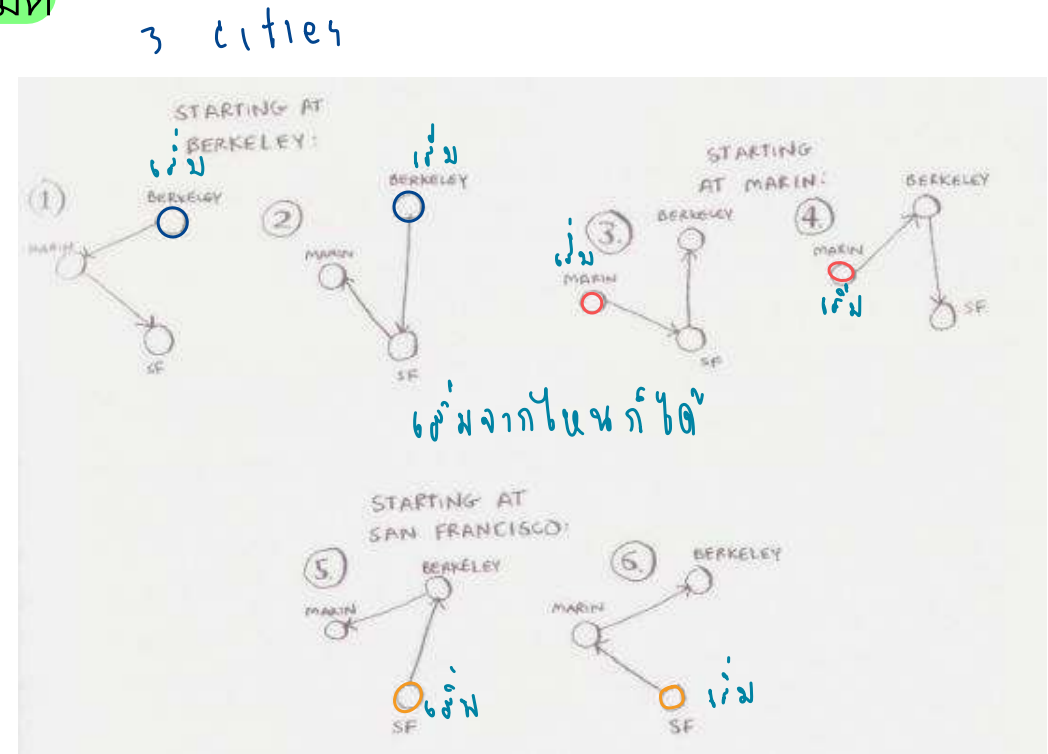
n = จาน. เมืองที่ไปจากของ



- การแก้ปัญหาทำได้โดย
 - หาเส้นทางการเดินทางของเซลล์แมนที่เป็นไปได้ทั้งหมด
 - เลือกเส้นทางที่เสียเวลาหรือค่าใช้จ่ายน้อยที่สุด

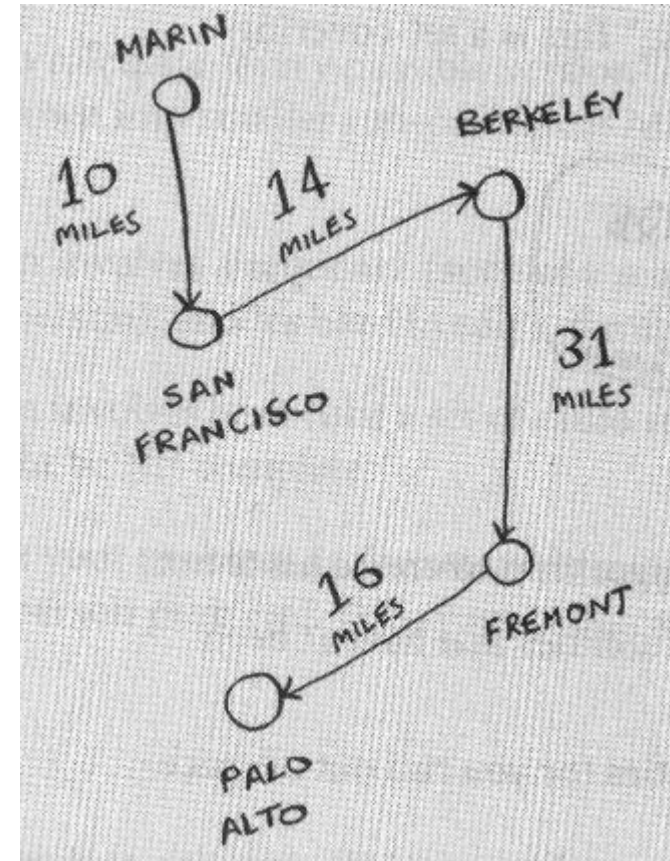
$O(n!)$

NUMBER OF CITIES	
1	→ 1 ROUTE
2	→ 2 START CITIES * 1 ROUTE FOR EACH START = 2 TOTAL ROUTES
3	→ 3 START CITIES * 2 ROUTES = 6 TOTAL ROUTES
4	→ 4 START CITIES * 6 ROUTES = 24 TOTAL ROUTES
5	→ 5 START CITIES * 24 ROUTES = 120 TOTAL ROUTES



TRAVELING SALESMAN PROBLEM

- การแก้ปัญหาด้วย Greedy algorithm
 - กำหนดจุดเริ่มต้น
 - เลือกเดินทางจากจุดเริ่มต้นไปยังเมืองที่ยังไม่เคยไปและใกล้ที่สุดก่อน
 - ทำเช่นนี้ไปเรื่อยๆ จนครบทุกเมือง

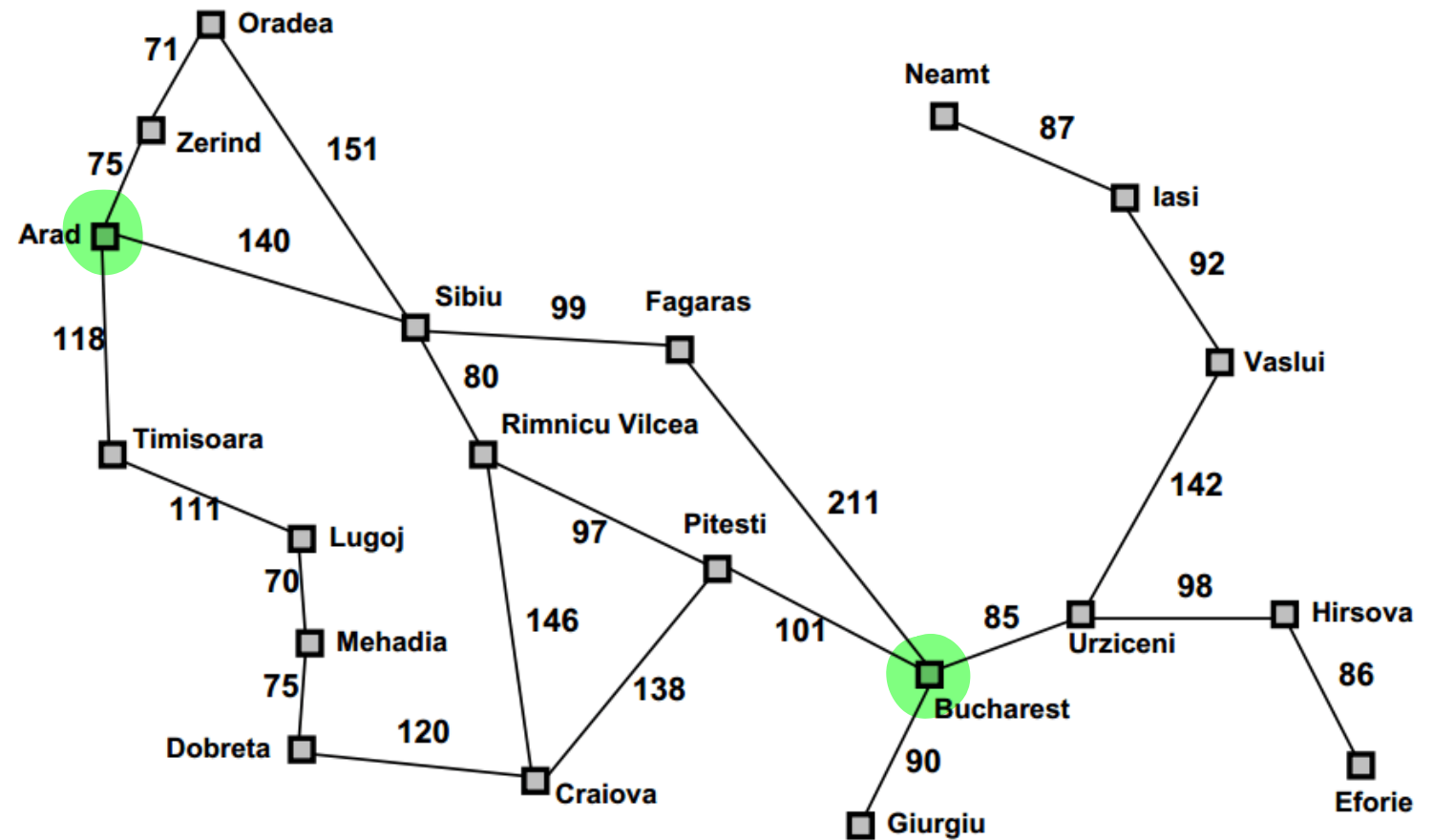


EXERCISE 1

- จงแสดงขั้นตอนการหาเส้นทางจากเมือง Arad ไปยังเมือง Bucharest ด้วย Greedy Algorithm
 - อธิบายแนวคิดในการเลือกเส้นทางที่ดี
 - สรุปเส้นทางผลลัพธ์ว่าต้องเดินผ่านเมืองใดบ้าง
 - เส้นทางที่หาได้นี้ เป็นคำตอบที่ดีที่สุดหรือไม่ (ใช้ระยะทางในการเดินทางสั้นที่สุด)

EXERCISE 1

- แผนที่ Romania



EXERCISE 2

- สมมติว่าคุณเดินทางไปประเทศอังกฤษ และมีเวลาท่องเที่ยวเป็นเวลา 3 วัน ให้คุณใช้ Greedy Algorithm วางแผนจะไปสถานที่ท่องเที่ยวต่างๆ โดยที่
 - (2.1) พยายามให้ได้ผลรวมเรตติ้งของสถานที่ที่ไปมากที่สุด
 - (2.2) พยายามให้ได้จำนวนสถานที่ท่องเที่ยวมากที่สุด

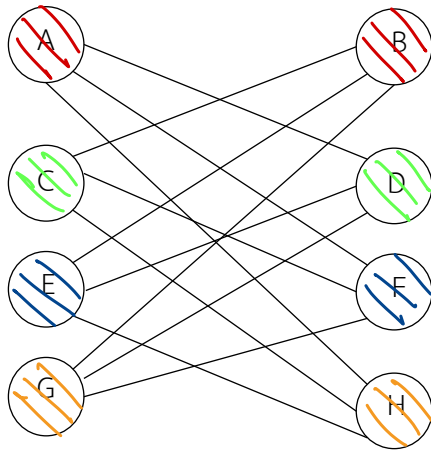
- จงเขียนแนวคิดในการเลือกทางแก้ปัญหาที่ดีที่สุด
- ขั้นตอนการเลือกสถานที่ท่องเที่ยว
- สรุปด้วยว่าคำตอบที่ได้เป็นคำตอบที่ดีที่สุดหรือไม่ (ถ้าไม่ใช่ ให้เขียนด้วยว่าคำตอบที่ดีที่สุดคืออะไร)

ATTRACTION	TIME	RATING
WESTMINSTER ABBEY	1/2 DAY	7
GLOBE THEATER	1/2 DAY	6
NATIONAL GALLERY	1 DAY	9
BRITISH MUSEUM	2 DAYS	9
ST. PAUL'S CATHEDRAL	1/2 DAY	8

EXERCISE 3

- จงใช้ Greedy Algorithm แก้ปัญหาการระบายสีกราฟ โดยห้ามระบายเวอร์เท็กซ์ที่เชื่อมต่อกันด้วยสีเดียวกัน โดยพยายามใช้สีให้น้อยที่สุด → *ห้บงกจ น. ล*
- จงอธิบายแนวคิดที่ใช้ในการแก้ปัญหา
- จงใช้แนวคิดดังกล่าว แสดงขั้นตอนการระบายสีกราฟดังรูป
 - กำหนดให้ระบายสีเวอร์เท็กซ์ตามลำดับตัวอักษร ได้แก่ เริ่มระบายสีเวอร์เท็กซ์ A, B, C, ...
 - ลำดับสีที่ระบาย คือ สีแดง เขียว น้ำเงิน เหลือง

*4 สี
Optimal*



*2 สี
Optimal*

